

Modelling and analysis of a cyber-physical system now with monads

Practical assignment 2 of the Cyber-Physical Programming course

Ana Sá Oliveira pg60236

May 23, 2026

Contents

1	Introduction	3
2	First task	4
2.1	Code corrections	4
2.2	Function getTimeAdv	4
2.3	Function allValidPlays	4
2.3.1	Function advsWithLantern	5
2.3.2	Function allValidPlays	5
2.4	Function exec	6
2.5	Function leq17	7
2.5.1	Function gFinal	7
2.5.2	Function leq17	8
2.6	Function l17	9
2.6.1	Function reachableDurStates	9
2.6.2	Function l17	11
2.7	Conclusions	12
3	Second task	13
3.1	Function play	13
3.1.1	Function probTime	13
3.1.2	Function play	13
3.1.3	Ord State	14
3.2	Function plays	15
3.3	Conclusions	16
3.3.1	Function distExample	16
3.3.2	Interpretation of the results	16
4	Third task	18
4.1	Interactive Game Using the IO Monad	18
4.1.1	Function game	18
4.1.2	Function gameLoop	19
4.1.3	Function pickChoice	19
4.1.4	Function showMove	20
4.1.5	Function applyMove	20
4.1.6	Function drawDurationState	21
4.1.7	Function drawLeftSide	21
4.1.8	Function drawRightSide	21
4.1.9	Function drawTimerBar	22
4.1.10	Running the Game	22
4.1.11	Gameplay Examples	22
4.2	State Sequence of Adventurers' Movements	29
4.2.1	Monad ListDurLog	29
4.2.2	Function playLog	29
4.2.3	Function playsLog	30

4.2.4	Function diffStates	30
4.2.5	Function crossTime	31
4.2.6	Function showMove'	31
4.2.7	Function showPlaysLog	31
4.2.8	Function showPlaysLog2	32
4.2.9	Function allValidPlaysLog	33
4.2.10	Function execLog	34
4.2.11	Function bestPaths	34
4.2.12	Function statesToMoves	34
4.2.13	Function printBestPath	35
4.2.14	Function printBestPath2	35
4.3	Conclusions	37

5 Conclusion 39

Chapter 1

Introduction

This is the report of the second practical assignment of the Cyber-Physical Programming course. The goal of this assignment is to complete 3 tasks related to the modelling and analysis of a cyber-physical system using monads.

The assignment is centred around the Bridge and Torch problem, a classic puzzle in which four adventurers must cross a bridge at night with a single torch. The bridge can hold at most two people at a time, and the torch must always accompany whoever is crossing. Each adventurer has a different crossing time, and when two cross together the slower one determines the total time. The goal is to get everyone across in at most 17 minutes. A skeleton implementation in Haskell was provided, including the base types and some initial functions, which we extended and corrected throughout the three tasks.

Chapter 2

First task

2.1 Code corrections

First, I noticed a mistake in the function **changeState** that was given in the assignment. The function was defined as follows:

```
changeState :: Objects -> State -> State
changeState o s = \x -> if x == o then not (s o) else s o
```

However, when x is not equal to o , the function should return the value of $s\ x$ instead of $s\ o$. We want to change the state of the object o , but we want to keep the state of all other objects unchanged. Therefore, the correct definition of the function is as follows:

```
changeState :: Objects -> State -> State
changeState o s = \x -> if x == o then not (s o) else s x
```

2.2 Function getTimeAdv

The first thing I had to do in the first task was to define the function **getTimeAdv** that given an adventure, returns the time that the adventure takes to cross the bridge. This function is defined as follows:

```
getTimeAdv :: Adventure -> Time
getTimeAdv P1 = 1
getTimeAdv P2 = 2
getTimeAdv P5 = 5
getTimeAdv P10 = 10
```

P1 is the adventurer that takes 1 minute to cross the bridge, P2 is the adventurer that takes 2 minutes to cross the bridge, P5 is the adventurer that takes 5 minutes to cross the bridge and P10 is the adventurer that takes 10 minutes to cross the bridge.

2.3 Function allValidPlays

The second thing I had to do was to define the function **allValidPlays** that given a state of the game, returns all possible moves that the adventurers can make.

To do this, I first defined one auxiliary function.

2.3.1 Function `advsWithLantern`

The function `advsWithLantern` receives a state of the game and returns a list of adventurers that are on the same side as the lantern. It returns the adventures that are with the lantern, the adventurers that can cross the bridge and therefore the adventurers that can be part of a valid move. This function is defined as follows:

```
advsWithLantern :: State -> [Adventurer]
advsWithLantern s = filter (\a -> s (Left a) == s(Right ())) [P1, P2, P5, P10]
```

We filter the list of all adventurers to get only those that are on the same side as the lantern (being on the same side as the lantern means that the state of the adventurer is the same as the state of the lantern).

2.3.2 Function `allValidPlays`

The function `allValidPlays` is defined as follows:

```
allValidPlays :: State -> ListDur State
allValidPlays s = manyChoice (singles ++ pairs)

    where singles    = [ LD [Duration
                            (getTimeAdv a,
                             mChangeState [Left a, Right ()] s)]
                        | a <- (advsWithLantern s) ]

    pairs            = [ LD [Duration
                            (max (getTimeAdv a1) (getTimeAdv a2),
                             mChangeState [Left a1, Left a2, Right ()] s)]
                        | (a1, a2) <- makePairs (advsWithLantern s) ]
```

The function `allValidPlays` generates all valid moves that the adventurers can make from a given state of the game.

This function starts to generate two lists of moves:

- **singles** - a list of moves where only one adventurer crosses the bridge.
- **pairs** - a list of moves where two adventurers cross the bridge together.

First, we generate the list **singles**. We consider all adventurers that are on the same side as the lantern, obtained by using `advsWithLantern s`. For each adventurer `a`, we create a possible move where that adventurer crosses the bridge alone with the lantern. The time of the move is given by `getTimeAdv a`, which corresponds to the individual crossing time of the adventurer. The resulting state is computed using `mChangeState [Left a, Right ()] s`, which updates the position of the adventurer and moves the lantern to the opposite side.

This results in a list of `ListDur State`, where each `ListDur` contains exactly one element representing a single-adventurer move. Therefore, this list represents all possible and valid single-adventurer moves that the adventurers can make in the current state.

Then we generate the list **pairs**. We consider all possible pairs of adventurers that are on the same side as the lantern, obtained by using `makePairs (advsWithLantern s)`. For each pair `(a1, a2)`, we create a possible move where both adventurers cross the bridge together with the lantern. The time of the move is determined by the slower adventurer, computed as `max (getTimeAdv a1) (getTimeAdv a2)`, since both must cross at the same pace. The resulting state is computed using `mChangeState [Left a1, Left a2, Right ()] s`, which updates the position of both adventurers and moves the lantern to the opposite side.

This results in a list of `ListDur State`, where each `ListDur` contains exactly one element representing a two-adventurer move. Therefore, this list represents all possible and valid pair moves that the adventurers can make in the current state.

Finally, both lists **singles** and **pairs** are combined using list concatenation, `singles ++ pairs`, and the function `manyChoice` is applied to the result.

The function `manyChoice` takes a list of values of type `ListDur a` and combines them into a single `ListDur a`. Each element in the input list represents an independent set of possible computations, each with associated durations. The function works by first removing the `ListDur` constructor from each element using `remLD`, which exposes the underlying lists of `Duration a`. It then concatenates all these lists into a single list using `concat`, effectively merging all possible alternatives. Finally, the result is wrapped again using the `LD` constructor to reconstruct the `ListDur` structure.

```
manyChoice :: [ListDur a] -> ListDur a
manyChoice = LD . concat . (map remLD)
```

Functions very similar to `manyChoice` appear frequently in similar exercises because they capture a common pattern in functional programming: the need to combine multiple independent monadic computations into a single structure representing all possible outcomes. This is especially common in problems involving non-determinism or search, where multiple alternative results are generated and must be unified into a single computation that represents **the full space of possibilities**.

Thus, by combining all independent computations into a single structure, we obtain a complete representation of **all valid moves available from the current state**. This ensures that every valid single-adventurer and pair-adventurer action is included, allowing the system to fully capture the set of all valid moves in the bridge crossing problem.

2.4 Function `exec`

The function `exec` receives an integer `n` and an initial state, and computes all possible sequences of `n` moves that the adventurers can perform from that state. Each sequence of moves produces a corresponding final state together with the total accumulated duration of all moves in that sequence, and it is precisely these final states paired with their respective durations that are generated by the function.

```
exec :: Int -> State -> ListDur State
exec 0 s = return s
```

```
exec n s = do s' <- allValidPlays s
           exec (n-1) s'
```

The first line of the function, `exec 0 s = return s`, defines **the base case of the recursion**. When the number of remaining moves is 0, no further actions are performed and the current state is returned unchanged with zero accumulated duration.

In this context, `return` wraps a plain value of type `State` into the computational structure `ListDur State`. In monadic terms, it corresponds to the unit operation, which embeds a value into the computational context without introducing additional effects or transitions. In this context, the state is embedded into the computational context without introducing any additional effects, meaning it remains unchanged and is associated with a zero duration.

```
instance Applicative ListDur where
  pure x = LD [Duration (0,x)]
  ...
instance Monad ListDur where
  return = pure
  ...
```

The remainder of the function defines the **recursive case**.

In the recursive case, `allValidPlays s` generates a value of type `ListDur State`, containing all possible next states together with their associated durations. Through the expression `s' <- allValidPlays s`, each generated `State` is individually passed to the recursive call `exec (n-1) s'`, allowing all possible execution paths to be explored. Although the recursion explicitly manipulates only states, the associated durations are not lost; they are automatically preserved and accumulated by the `ListDur` monad through the bind operation (`>>=`), which is implicitly used by the `do` notation.

```
instance Monad ListDur where
  ...
l >>= k = LD $ do x <- remLD l
                  g x where
                    g(Duration (i,x)) = let u = (remLD (k x))
                                          in map (\(Duration (i',x)) -> Duration (i + i', x)) u
```

As a result, the function produces a final value of type `ListDur State`, containing all reachable states after `n` moves together with their corresponding accumulated durations.

2.5 Function `leq17`

To implement this function, first we introduced an auxiliary function.

2.5.1 Function `gFinal`

The function `gFinal` represents the goal state of the game, where every adventurer has reached the final side. It is defined as a constant function (`const True`). This reflects the fact that, in this state, all objects in the game are assumed to be on the final side of the bridge, regardless of their identity.


```
gFinal :: State
gFinal = const True
```

This function is the opposite of **gInit**, which is defined as `const False`, representing the initial state of the game where all objects are assumed to be on the starting side of the bridge.

```
gInit :: State
gInit = const False
```

2.5.2 Function `leq17`

The function **leq17** determines whether it is possible for all adventurers to reach the other side in at most 17 minutes, without exceeding 5 moves. The function takes no input arguments and returns a boolean value, indicating whether the condition is satisfied or not.

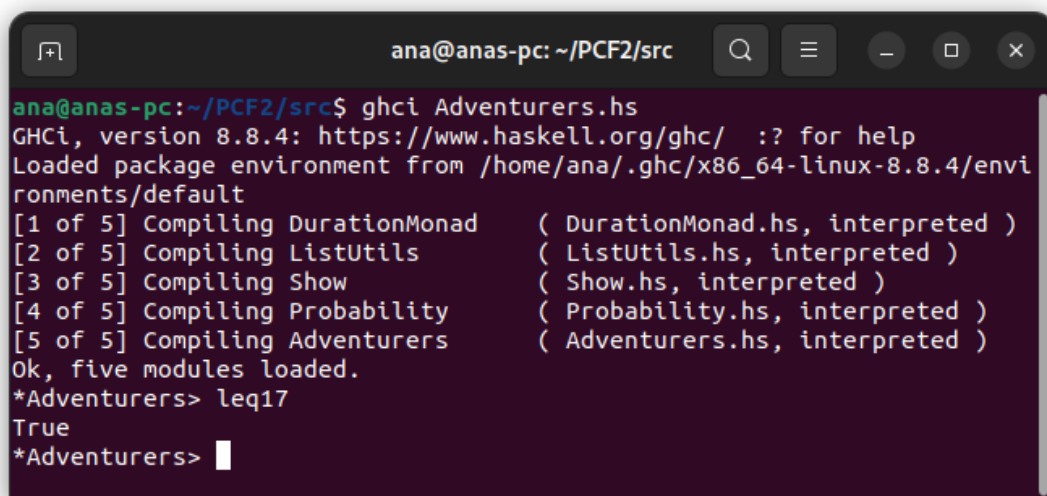
```
leq17 :: Bool
leq17 = any (\(Duration (t,s)) -> s == gFinal && t <= 17)
  (remLD (manyChoice [exec n gInit | n <- [0..5]]))
```

The expression `[exec n gInit | n <- [0..5]]` generates all reachable states without exceeding 5 moves, together with their corresponding accumulated durations.

The function **manyChoice** takes the resulting list of **ListDur** computations and combines them into a single **ListDur**, collecting all possible outcomes into one non-deterministic computation.

The function **remLD** is then used to extract the underlying list of **Duration State** values from the **ListDur** structure, making the individual results directly accessible.

Finally, **any** is applied to this list to check whether there exists at least one execution for which the final state corresponds to **gFinal** and the total accumulated time does not exceed 17 minutes. If such a case exists, the function returns `True`; otherwise, it returns `False`.



```
ana@anas-pc: ~/PCF2/src
ana@anas-pc:~/PCF2/src$ ghci Adventurers.hs
GHCi, version 8.8.4: https://www.haskell.org/ghc/  :? for help
Loaded package environment from /home/ana/.ghc/x86_64-linux-8.8.4/environments/default
[1 of 5] Compiling DurationMonad (DurationMonad.hs, interpreted)
[2 of 5] Compiling ListUtils (ListUtils.hs, interpreted)
[3 of 5] Compiling Show (Show.hs, interpreted)
[4 of 5] Compiling Probability (Probability.hs, interpreted)
[5 of 5] Compiling Adventurers (Adventurers.hs, interpreted)
Ok, five modules loaded.
*Adventurers> leq17
True
*Adventurers> 
```

Figure 2.1: Evaluation of the function `leq17` in GHCi.

We evaluated `leq17` in GHCi using the `Adventurers` module, and the result returned was **True**, indicating that it is possible for all adventurers to reach the other side in at most 17 minutes, without exceeding 5 moves.

2.6 Function `l17`

The function `l17` determines whether it is possible for all adventurers to reach the other side in strictly less than 17 minutes. The function takes no input arguments and returns a boolean value, indicating whether the condition is satisfied or not.

In this case, we aim to verify this property for any number of moves. If we proceeded as before, the function would not terminate in the case where the condition is false, since the search space would be infinite. Therefore, we need to change our approach. We require auxiliary functions to proceed.

2.6.1 Function `reachableDurStates`

The function `reachableDurStates` computes all states reachable from an initial state, together with the accumulated duration required to reach each state. Only states whose accumulated duration is strictly less than the given limit are included.

The function performs a breadth-first exploration of the state space, starting from the initial state with duration zero. Each transition is extended only if the resulting accumulated duration does not exceed the specified bound and if the resulting state-duration pair has not been visited before.

```
reachableDurStates :: Int -> State -> ListDur State
reachableDurStates limit s0 = LD (go [Duration (0, s0)] [(s0, 0)])
  where
    go [] _ = []

    go (Duration (t, s) : ws) seen =
      Duration (t, s) : go (ws ++ newNexts) seen'
    where
      nexts = remLD (allValidPlays s)

      newNexts =
        [ Duration (t + dt, s')
        | Duration (dt, s') <- nexts
        , t + dt < limit
        , notElem (s', t + dt) seen
        ]

    seen' =
      map (\(Duration (t', s')) -> (s', t')) newNexts ++ seen
```

After the definition of the function, the algorithm is implemented through a recursive breadth-first search encoded in the auxiliary function `go`.

The initial call creates the initial search frontier and the visited memory structure:

```
go [Duration (0, s0)] [(s0, 0)]
```

The frontier is a list of `Duration (t, s)` values, where `t` represents the accumulated time and `s` the current state. Initially, the search starts at time zero with the initial state `s0`.

The second argument, `seen`, is a list of pairs `(State, Time)` used to record which state-time combinations have already been explored. This structure prevents revisiting the same configuration at the same or worse time.

The recursive function `go` is defined as follows:

```
go [] _ = []
```

This base case states that when the frontier becomes empty, the search terminates.

The main recursive step processes the next state in the frontier:

```
go (Duration (t, s) : ws) seen =
```

Here, `Duration (t, s)` represents the current node being expanded, while `ws` is the remaining frontier.

The current state is immediately included in the output:

```
Duration (t, s) : go (ws ++ newNexts) seen'
```

This ensures a full traversal of all reachable states in exploration order. The frontier is updated by appending newly generated successor states (`newNexts`) to the remaining worklist `ws`.

Successor states are generated using:

```
nexts = remLD (allValidPlays s)
```

The function `allValidPlays` produces a non-deterministic list of all possible transitions from the current state `s`. The function `remLD` extracts this structure into a standard list of `Duration (dt, s')` values.

New candidate states are then constructed using:

```
newNexts =
  [ Duration (t + dt, s')
  | Duration (dt, s') <- nexts
  , t + dt < limit
  , notElem (s', t + dt) seen
  ]
```

Each expression plays a specific role:

- `Duration (dt, s') <- nexts` iterates over all possible transitions from the current state.
- `t + dt < limit` enforces the global constraint that only states reachable within the time bound are considered.

- `notElem (s', t + dt) seen` ensures that the algorithm does not revisit already explored state-time pairs, avoiding redundant exploration and infinite loops.
- `Duration (t + dt, s')` constructs the new reachable configuration with updated accumulated time.

The visited structure is updated as follows:

```
seen' =
  map (\(Duration (t', s')) -> (s', t')) newNexts ++ seen
```

This expression extracts the state-time pairs from the newly generated nodes and appends them to the previously visited list. The use of `map` converts each `Duration (t', s')` into the pair `(s', t')`, ensuring consistency with the representation used in `seen`.

Finally, the recursive call:

```
go (ws ++ newNexts) seen'
```

continues the exploration by expanding the frontier with newly discovered states while carrying forward the updated visited set. The algorithm terminates when no further states can be generated within the time limit, ensuring that all reachable configurations are explored under the given constraint.

2.6.2 Function l17

Again, the function `l17` determines whether it is possible for all adventurers to reach the other side in strictly less than 17 minutes. The function takes no input arguments and returns a boolean value, indicating whether the condition is satisfied or not.

```
l17 :: Bool
l17 = any (\(Duration (t,s)) -> s == gFinal)
  (remLD (reachableDurStates 17 gInit))
```

The expression `reachableDurStates 17 gInit` computes all states reachable from the initial state `gInit` whose accumulated time is strictly less than 17 minutes. The result is a `ListDur State`, where each element contains a state and the corresponding accumulated time required to reach it.

The function `remLD` is then used to extract the underlying list of `Duration State` values from the `ListDur` structure, making the reachable states directly accessible for further processing.

Finally, the function `any` is applied to this list to check whether there exists at least one reachable configuration in which all adventurers have reached the final state `gFinal` within the given time limit.

If such a state exists, the function returns `True`, meaning that it is possible for all adventurers to cross the bridge in strictly less than 17 minutes. Otherwise, it returns `False`, indicating that no valid sequence of moves satisfies the time constraint.

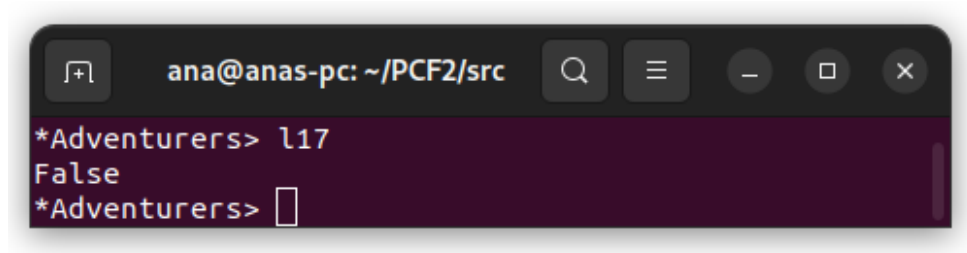
A screenshot of a terminal window with a dark background. The window title bar shows the user 'ana@anas-pc' and the directory '~/PCF2/src'. The terminal content shows a prompt '*Adventurers>' followed by the command 'l17', which returns 'False'. A second prompt '*Adventurers>' is shown with a cursor. The terminal has standard window controls (search, menu, zoom, close) in the title bar.

Figure 2.2: Evaluation of the function `l17` in GHCi.

We evaluated `l17` in GHCi using the `Adventurers` module, and the result returned was **False**, confirming that it is impossible for all adventurers to cross the bridge in less than 17 minutes.

2.7 Conclusions

We completed the first task as follows:

- We modeled the system using Haskell, based on monadic structures, in particular the duration and non-deterministic monads. This was achieved by correcting the code and implementing the functions `getTimeAdv`, `allValidPlays`, `exec`, etc.
- We showed that it is possible for all adventurers to reach the other side in at most 17 minutes. Using the function `leq17`, we showed that there exists a solution achieving this in at most 5 moves. Therefore, we conclude that it is possible.
- We showed that it is impossible for all adventurers to reach the other side in strictly less than 17 minutes, using the function `l17`.

Chapter 3

Second task

3.1 Function play

The function **play** calculates the resulting state based on which adventurers to move next and their probabilistic crossing times. To implement this function, first we introduced an auxiliary function.

3.1.1 Function probTime

The function **probTime** replaces a deterministic crossing time x with a uniform distribution over three possible crossing times: $x - \frac{x}{2}$, x , and $x + \frac{x}{2}$.

```
probTime :: Int -> Dist Int
probTime x = uniform [x - x `div` 2, x, x + x `div` 2]
```

A uniform distribution assumes that all values are equally likely to occur, so in this case each crossing time has probability $\frac{1}{3}$. This introduces uncertainty into the model by allowing the crossing time to vary around its original value.

3.1.2 Function play

The function **play** calculates the resulting state based on which adventurers to move next and their probabilistic crossing times.

```
play :: Move -> State -> DistDur State
play (Left a) s =
  DD (do t <- probTime (getTimeAdv a)
        return (Duration (t, mChangeState [Left a, Right ()] s)))
play (Right (a1, a2)) s =
  DD (do t1 <- probTime (getTimeAdv a1)
        t2 <- probTime (getTimeAdv a2)
        return (Duration (max t1 t2, mChangeState [Left a1, Left a2, Right ()] s)))
```

The function **play** takes a **Move** and the current **State** as input and returns a **DistDur State**, representing a probability distribution of crossing times associated with the single resulting state.

The function **play** can take two different types of **Move**: either a single adventurer moving (**Left a**) or a pair of adventurers moving together (**Right (a1, a2)**).

When the function receives **Left a**, it represents a move where a single adventurer crosses. The function first obtains the adventurer's deterministic crossing-time using **getTimeAdv a**. Then, it applies the **probTime** function to this value, resulting in a distribution of possible crossing times for that adventurer. For every possible crossing time **t** generated

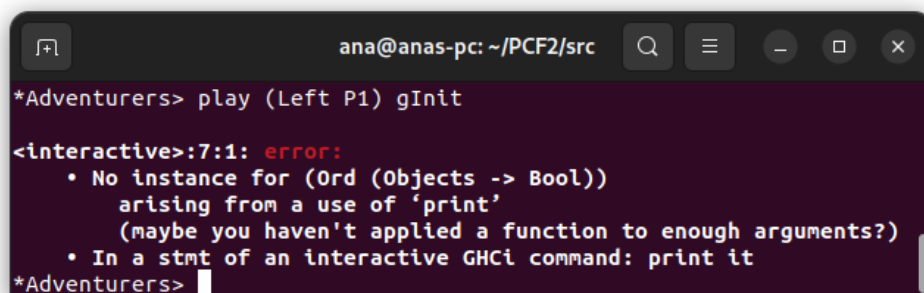
by the distribution, the function constructs a `Duration` value that pairs this time `t` with the resulting state after the move, computed using `mChangeState [Left a, Right ()]` `s`. Then it wraps this in a `return` to lift it into the `Dist` monad, and finally wraps the entire computation in `DD` to produce a `DistDur State`.

When the function receives `Right (a1, a2)`, it represents a move where two adventurers cross together. The function first obtains the deterministic crossing times for both adventurers using `getTimeAdv a1` and `getTimeAdv a2`. Then, it applies the `probTime` function to each of these values, resulting in two distributions of possible crossing times. For every possible pair of crossing times `t1` and `t2` generated by these distributions, the function constructs a `Duration` value that pairs the maximum of these two times `max t1 t2` (since both adventurers must cross together at the pace of the slower one), with the resulting state after the move, computed using `mChangeState [Left a1, Left a2, Right ()]` `s`. Then it wraps this in a `return` to lift it into the `Dist` monad, and finally wraps the entire computation in `DD` to produce a `DistDur State`.

This function effectively returns a probability distribution of the crossing times for the given move, along with the resulting state after performing that move.

3.1.3 Ord State

However, when executing `play` with valid arguments, the program produces a runtime error when attempting to display the result in the terminal.



```

ana@anas-pc: ~/PCF2/src
*Adventurers> play (Left P1) gInit

<interactive>:7:1: error:
  • No instance for (Ord (Objects -> Bool))
    arising from a use of 'print'
    (maybe you haven't applied a function to enough arguments?)
  • In a stmt of an interactive GHCi command: print it
*Adventurers>

```

Figure 3.1: Runtime error encountered during execution of `play`.

This occurs because printing a value of type `DistDur State` requires converting a probabilistic distribution into a human-readable form. The `Show` instance for distributions performs normalization, which involves sorting the elements of the distribution. This sorting requires an `Ord` instance for the underlying type. Since `State` is defined as a function `(Objects -> Bool)`, it does not admit an `Ord` instance in Haskell, leading to the error.

We solved this issue by manually defining an `Ord State` instance based on the string representation of states:

```

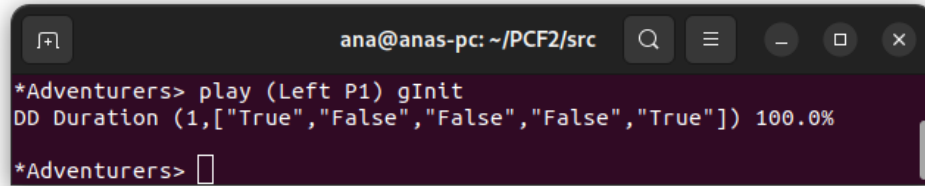
instance Ord State where
  compare s1 s2 = compare (show s1) (show s2)

```

This works by converting each state (a function) into a `String` using `show`, and then comparing these strings lexicographically. This allows the distribution to be printed

without requiring a true semantic ordering on functions. However, this ordering is only used for display purposes and does not represent a meaningful ordering of states.

After this change, the function works correctly.



```

ana@anas-pc: ~/PCF2/src
*Adventurers> play (Left P1) gInit
DD Duration (1,["True","False","False","False","True"]) 100.0%
*Adventurers> 

```

Figure 3.2: Evaluation of the function `play` in GHCi with sample arguments.

3.2 Function plays

The function **plays** extends the previous function (function `play`) to lists of movements. It receives a list of moves and an initial state, and computes the list of moves sequentially, starting from the initial state. Each sequence of moves produces a `DistDur State`.

```

plays :: [Move] -> State -> DistDur State
plays []      s = return s
plays (m:ms) s = do s' <- play m s
                  plays ms s'

```

The first line of the function, `plays [] s = return s`, defines **the base case of the recursion**. When the list of moves is empty, no further actions are performed and the current state is returned unchanged without introducing additional effects or transitions. In this context, `return` wraps a plain value of type `State` into the computational structure `DistDur State`. In monadic terms, it corresponds to the unit operation, which embeds a value into the computational context without introducing additional effects or transitions.

The remainder of the function defines the **recursive case**. In each recursive step, the function takes the first move `m` from the list `(m:ms)` together with the current state `s`. The expression `play m s` executes that move and produces a value of type `DistDur State`, representing a distribution of possible resulting states. Each resulting state is associated not only with a duration, but also with a probability that reflects the likelihood of that transition occurring. Using the binding expression `s' <- play m s`, the computation conceptually iterates over every possible state produced by the distribution and passes each one to the recursive call `plays ms s'`. Thus, the remaining sequence of moves `ms` is executed independently for every possible intermediate state generated after the first move. This recursive structure allows the computation to explore all possible execution paths of the sequence of moves. Although the recursion explicitly manipulates only values of type `State`, both the probabilities and the durations associated with each transition are automatically preserved and combined by the `DistDur` monad through the bind operation (`>>=`), which is implicitly used by the `do` notation. During this process, probabilities are propagated and combined along the different execution paths, while durations are accumulated across consecutive moves.

As a result, the final value produced by `plays` represents a complete distribution over all reachable final states after executing the entire list of moves, together with the probability and total duration associated with each possible execution path.

The **strong similarity** between the function `plays` and the function `exec` from the previous task highlights **one of the main advantages of monads**: they provide a uniform structure for sequencing recursive computations while abstracting the management of computational effects. Although the two functions operate over **different monads** and model different behaviors — probabilistic distributions in `DistDur` and nondeterministic exploration in `ListDur` — their **recursive logic remains essentially identical** because the propagation of probabilities, branching, and duration accumulation are handled automatically by the monadic bind operation.

3.3 Conclusions

In the second task, we implemented all the required functions. It now remains to answer the following question:

In this probabilistic setting, what can you conclude about the adventurer's claims mentioned above?

One adventurer claims that they **cannot be all on the other side in less than 19 minutes**. Another one disagrees and claims that it **can be done in 17 minutes**.

To answer this question, we implemented an auxiliary function.

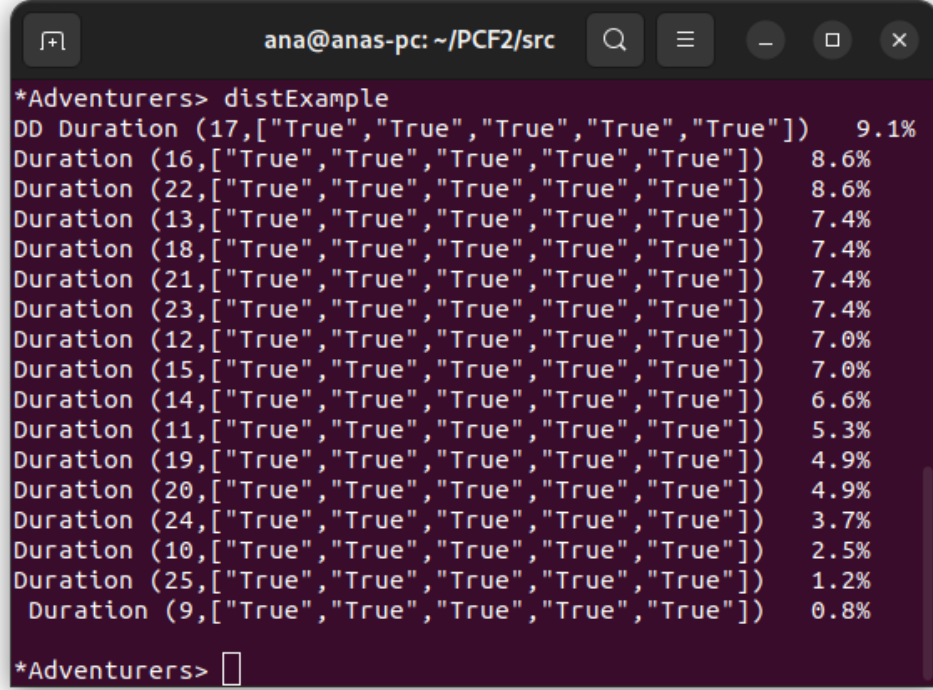
3.3.1 Function `distExample`

The function `distExample` returns one distribution over all reachable final states after executing one specific list of moves, together with the probability and total duration associated with each possible execution path (since we use the function `plays`). The chosen sequence of moves corresponds to an optimal solution that was easily found online, namely: moving P1 and P2 forward, moving P1 back, moving P5 and P10 forward, moving P2 back, and finally moving P1 and P2 forward. The initial state is the initial state of the game (`gInit`).

```
distExample :: DistDur State
distExample = plays
[Right (P1,P2), Left P1, Right (P5,P10), Left P2, Right (P1,P2)] gInit
```

3.3.2 Interpretation of the results

Running `distExample` yields the following distribution:



```
*Adventurers> distExample
DD Duration (17,["True","True","True","True","True"]) 9.1%
Duration (16,["True","True","True","True","True"]) 8.6%
Duration (22,["True","True","True","True","True"]) 8.6%
Duration (13,["True","True","True","True","True"]) 7.4%
Duration (18,["True","True","True","True","True"]) 7.4%
Duration (21,["True","True","True","True","True"]) 7.4%
Duration (23,["True","True","True","True","True"]) 7.4%
Duration (12,["True","True","True","True","True"]) 7.0%
Duration (15,["True","True","True","True","True"]) 7.0%
Duration (14,["True","True","True","True","True"]) 6.6%
Duration (11,["True","True","True","True","True"]) 5.3%
Duration (19,["True","True","True","True","True"]) 4.9%
Duration (20,["True","True","True","True","True"]) 4.9%
Duration (24,["True","True","True","True","True"]) 3.7%
Duration (10,["True","True","True","True","True"]) 2.5%
Duration (25,["True","True","True","True","True"]) 1.2%
Duration (9,["True","True","True","True","True"]) 0.8%

*Adventurers> 
```

Figure 3.3: Evaluation of the function `distExample` in GHCi.

From the obtained distribution, we observe that all execution paths reach the final state where all adventurers are on the other side. However, the total duration varies across different executions due to the probabilistic nature of the model.

Importantly, the results show that there exist valid execution paths with a total duration of 17 minutes. This means that completing the task in 17 minutes is possible, but not guaranteed in every execution.

On the other hand, we also observe executions with durations both below and above 19 minutes. This implies that the statement claiming that it is impossible to complete the task in less than 19 minutes is incorrect, since there are valid paths with smaller durations.

It is also worth noting that in this probabilistic setting, it is even possible—if we are “lucky” in the chosen transitions—for the task to be completed in less than 17 minutes. This contrasts with the model studied previously, where such a result was impossible.

Therefore, in this probabilistic setting, **the first adventurer’s claim is false**, since there exist valid execution paths where the task is completed in less than 19 minutes.

The second adventurer’s claim is true, because there are execution paths in which the task can be completed in 17 minutes. However, **although this is possible, it is not guaranteed that the execution will always take 17 minutes for the given sequence of moves, due to the probabilistic nature of the model.**

Chapter 4

Third task

In the third task, we were asked to add new functionalities to the code which we found essential or interesting.

I think it is interesting to implement an interactive game using the IO monad, where the user can choose the moves to perform and see the resulting state and duration after each move. This would allow us to explore the system in a more dynamic way and understand how different sequences of moves lead to different outcomes. I also think it would be fun to play the game and try to find the optimal sequence of moves to reach the goal state in the shortest time possible.

I think it is also essential to see the sequence of states respective to the movements of the adventurers from the initial state to the final goal. Without it, we can only observe the final state and the total duration, but not how each state was reached. By inspecting the full sequence of intermediate states, we can understand the role of each move, trace back the reasoning behind a given solution, confirm that the constraints of the problem are respected at every step, and most importantly know exactly which moves to perform and in what order — which is ultimately what it means to actually solve the problem. Observing the sequence of moves that lead to each pair of final state and duration is particularly important, as it allows us to understand how each state is reached and how different durations can be achieved.

In the following sections, I will briefly describe how these functionalities were implemented and what they consist of.

4.1 Interactive Game Using the IO Monad

The game is implemented using the IO monad to handle all input and output operations. This allows the program to interact with the player while maintaining a clear separation between pure game logic and side effects such as reading input or printing to the screen. The game follows a recursive loop structure, where each iteration displays the current state, processes user input, and updates the game accordingly until a terminal condition is reached (either success or failure).

4.1.1 Function game

The `game` function starts the execution of the program by calling the main game loop with the initial configuration.

```
game :: IO ()
game = do
  gameLoop (Duration 0, gInit))
```

This function initializes the game state wrapped inside the `Duration` type, which stores both the current state of the game and the elapsed time. From this point onward, control is delegated entirely to the `gameLoop` function.

4.1.2 Function `gameLoop`

The `gameLoop` function is the core of the game execution. It is responsible for rendering the current state, checking termination conditions, and generating available moves.

```
gameLoop :: Duration State -> IO ()
gameLoop ds@(Duration (t,s)) = do
  putStr "\ESC[2J\ESC[1H"
  putStrLn ""
  putStrLn " \ESC[35mBridge and Torch Game\ESC[0m | Save everyone in 17 minutes or less"
  drawDurationState ds
  if s == gFinal
  then do
    if t <= 17
    then do
      putStrLn " Mission: \ESC[32mCOMPLETED\ESC[0m -- you won!"
    else do
      putStrLn " Mission: \ESC[31mFAILED\ESC[0m -- try again!"
    pickChoice ds []
  else do
    let available = advsWithLantern s
        singles   = map Left available
        pairs     = map (\(a1,a2) -> Right (a1,a2)) (makePairs available)
        moves     = singles ++ pairs
    putStrLn " Mission: IN PROGRESS"
    pickChoice ds moves
```

This function determines whether the game has reached its final state. If all adventurers have successfully crossed the bridge, the game evaluates the total time and declares the result as either a success or failure. Otherwise, it computes the list of valid moves and continues the game.

4.1.3 Function `pickChoice`

The `pickChoice` function handles user interaction by displaying all available moves and processing the player's selection.

```
pickChoice :: Duration State -> [Move] -> IO ()
pickChoice ds moves = do
  putStrLn ""
  putStrLn " Available choices:"

  mapM_ (\(i,m) ->
    putStrLn $ " " ++ show i ++ ") " ++ showMove m
  ) (zip [1..] moves)
```

```

let restartChoiceNum = length moves + 1
    quitChoiceNum     = length moves + 2

putStrLn $ " " ++ show restartChoiceNum ++ ") [RESTART GAME]"
putStrLn $ " " ++ show quitChoiceNum ++ ") [QUIT GAME]"

putStrLn ""
putStr "  Choice: "
line <- getLine

case reads line of
  [(n,"")] | n >= 1 && n <= length moves ->
    gameLoop (applyMove (moves !! (n-1)) ds)

  [(n,"")] | n == restartChoiceNum ->
    gameLoop (Duration (0, gInit))

  [(n,"")] | n == quitChoiceNum ->
    putStr "\ESC[2J\ESC[1H"

  _ -> gameLoop ds

```

This function presents the player with all possible actions, including normal moves, restarting the game, or quitting. It then reads the player's input and updates the game state accordingly.

4.1.4 Function showMove

The `showMove` function is responsible for converting a move into a human-readable string representation, including the time cost of each action.

```

showMove :: Move -> String
showMove (Left a) =
  show a ++ " [" ++ show (getTimeAdv a) ++ " min]"
showMove (Right (a1,a2)) =
  show a1 ++ " + " ++ show a2 ++ " [" ++
  show (max (getTimeAdv a1) (getTimeAdv a2)) ++ " min]"

```

4.1.5 Function applyMove

The `applyMove` function applies a selected move to the current game state while updating the elapsed time accordingly.

```

applyMove :: Move -> Duration State -> Duration State
applyMove (Left a) (Duration (t,s)) =
  Duration ((getTimeAdv a) + t,
    mChangeState [Left a, Right ()] s)
applyMove (Right (a1,a2)) (Duration (t,s)) =
  Duration ((max (getTimeAdv a1) (getTimeAdv a2)) + t,
    mChangeState [Left a1, Left a2, Right ()] s)

```

This function encapsulates both the state transition and the time accumulation caused by each move.

4.1.6 Function drawDurationState

The `drawDurationState` function renders the full game state, including the positions of all characters and the elapsed time.

```
drawDurationState :: Duration State -> IO ()
drawDurationState (Duration (t,s)) = do
  putStrLn ""
  putStrLn "  -----not saved-----bridge-----saved-----"
  putStrLn ""
  putStrLn $ "  " ++ drawLeftSide s ++ "  -----" ++ drawRightSide s
  putStrLn ""
  putStrLn $ "  " ++ drawTimerBar t
  putStrLn ""
```

The bridge is represented using dashes (-) due to limitations in LaTeX regarding custom symbol rendering.

4.1.7 Function drawLeftSide

The `drawLeftSide` function displays the state of the left side of the bridge, including the torch and adventurers still waiting to cross.

```
drawLeftSide :: State -> String
drawLeftSide s =
  let torch = if not (s (Right ())) then "*" else "  "
      p1  = if not (s (Left P1))  then "P1 " else "  "
      p2  = if not (s (Left P2))  then "P2 " else "  "
      p5  = if not (s (Left P5))  then "P5 " else "  "
      p10 = if not (s (Left P10)) then "P10" else "  "
  in torch ++ p1 ++ p2 ++ p5 ++ p10
```

The symbol `*` is used to represent the light bulb emoji, as it could not be reliably rendered in LaTeX.

4.1.8 Function drawRightSide

The `drawRightSide` function displays the state of the right side of the bridge.

```
drawRightSide :: State -> String
drawRightSide s =
  let torch = if s (Right ()) then "*" else "  "
      p1  = if s (Left P1)  then "P1 " else "  "
      p2  = if s (Left P2)  then "P2 " else "  "
      p5  = if s (Left P5)  then "P5 " else "  "
      p10 = if s (Left P10) then "P10" else "  "
  in torch ++ p1 ++ p2 ++ p5 ++ p10
```

The symbol `*` is used to represent the light bulb emoji, as it could not be reliably rendered in LaTeX.

4.1.9 Function drawTimerBar

The `drawTimerBar` function visualizes the elapsed time and provides a simple progress indicator for the 17-minute constraint.

```
drawTimerBar :: Int -> String
drawTimerBar t =
    let barLength = min t 17
        dotsLength = 17 - barLength
        status = if t > 17
                    then " OVER 17!"
                    else " 17 min"
    in show t ++ " min " ++ replicate barLength '='
    ++ replicate dotsLength '.' ++ status
```

This function provides a visual representation of time progression, helping the player track performance relative to the optimal solution.

4.1.10 Running the Game

To run the game, open GHCi with the command `ghci Adventure.hs` and then execute the function `game`. The game will start immediately. Players then interact with the game by entering numeric choices to make decisions at each step.

4.1.11 Gameplay Examples

Below are two example runs of the game illustrating how the player interacts with the system and how decisions affect the game state.

Gameplay 1

This gameplay demonstrates a successful completion of the game, where the player wins by saving everyone within the 17-minute limit.

```
ana@anas-pc: ~/PCF2/src

Bridge and Torch Game – Save everyone in 17 minutes or less

-----not saved-----bridge-----saved-----

💡 P1 P2 P5 P10 ██████████

0 min ..... 17 min

Mission: IN PROGRESS

Available choices:
1) P1 [1 min]
2) P2 [2 min]
3) P5 [5 min]
4) P10 [10 min]
5) P2 + P1 [2 min]
6) P5 + P1 [5 min]
7) P5 + P2 [5 min]
8) P10 + P1 [10 min]
9) P10 + P2 [10 min]
10) P10 + P5 [10 min]
11) [RESTART GAME]
12) [QUIT GAME]

Choice: 5
```

```
ana@anas-pc: ~/PCF2/src

Bridge and Torch Game – Save everyone in 17 minutes or less

-----not saved-----bridge-----saved-----

P5 P10 ██████████ 💡 P1 P2

2 min == ..... 17 min

Mission: IN PROGRESS

Available choices:
1) P1 [1 min]
2) P2 [2 min]
3) P2 + P1 [2 min]
4) [RESTART GAME]
5) [QUIT GAME]

Choice: 1
```



```
ana@anas-pc: ~/PCF2/src

Bridge and Torch Game – Save everyone in 17 minutes or less

-----not saved-----bridge-----saved-----

💡 P1    P5 P10  [checkered bridge] P2

3 min ==..... 17 min

Mission: IN PROGRESS

Available choices:
1) P1 [1 min]
2) P5 [5 min]
3) P10 [10 min]
4) P5 + P1 [5 min]
5) P10 + P1 [10 min]
6) P10 + P5 [10 min]
7) [RESTART GAME]
8) [QUIT GAME]

Choice: 6
```

```
ana@anas-pc: ~/PCF2/src

Bridge and Torch Game – Save everyone in 17 minutes or less

-----not saved-----bridge-----saved-----

P1 [checkered bridge] 💡 P2 P5 P10

13 min =====... 17 min

Mission: IN PROGRESS

Available choices:
1) P2 [2 min]
2) P5 [5 min]
3) P10 [10 min]
4) P5 + P2 [5 min]
5) P10 + P2 [10 min]
6) P10 + P5 [10 min]
7) [RESTART GAME]
8) [QUIT GAME]

Choice: 1
```

```
ana@anas-pc: ~/PCF2/src

Bridge and Torch Game – Save everyone in 17 minutes or less

-----not saved-----bridge-----saved-----

💡 P1 P2          P5 P10

15 min =====.. 17 min

Mission: IN PROGRESS

Available choices:
1) P1 [1 min]
2) P2 [2 min]
3) P2 + P1 [2 min]
4) [RESTART GAME]
5) [QUIT GAME]

Choice: 3
```

```
ana@anas-pc: ~/PCF2/src

Bridge and Torch Game – Save everyone in 17 minutes or less

-----not saved-----bridge-----saved-----

          P1 P2 P5 P10

17 min ===== 17 min

Mission: COMPLETED -- you won!

Available choices:
1) [RESTART GAME]
2) [QUIT GAME]

Choice: 
```

Gameplay 2

This gameplay demonstrates an unsuccessful attempt, where the player fails to save everyone within the 17-minute limit and loses the game.

```
ana@anas-pc: ~/PCF2/src

Bridge and Torch Game – Save everyone in 17 minutes or less

-----not saved-----bridge-----saved-----

💡 P1 P2 P5 P10 ██████████

0 min ..... 17 min

Mission: IN PROGRESS

Available choices:
1) P1 [1 min]
2) P2 [2 min]
3) P5 [5 min]
4) P10 [10 min]
5) P2 + P1 [2 min]
6) P5 + P1 [5 min]
7) P5 + P2 [5 min]
8) P10 + P1 [10 min]
9) P10 + P2 [10 min]
10) P10 + P5 [10 min]
11) [RESTART GAME]
12) [QUIT GAME]

Choice: 7
```

```
ana@anas-pc: ~/PCF2/src

Bridge and Torch Game – Save everyone in 17 minutes or less

-----not saved-----bridge-----saved-----

P1 P10 ██████████💡 P2 P5

5 min ===== 17 min

Mission: IN PROGRESS

Available choices:
1) P2 [2 min]
2) P5 [5 min]
3) P5 + P2 [5 min]
4) [RESTART GAME]
5) [QUIT GAME]

Choice: 1
```

```
ana@anas-pc: ~/PCF2/src

Bridge and Torch Game – Save everyone in 17 minutes or less

-----not saved-----bridge-----saved-----

💡 P1 P2      P10  [10 blocks] P5

7 min ===== 17 min

Mission: IN PROGRESS

Available choices:
1) P1 [1 min]
2) P2 [2 min]
3) P10 [10 min]
4) P2 + P1 [2 min]
5) P10 + P1 [10 min]
6) P10 + P2 [10 min]
7) [RESTART GAME]
8) [QUIT GAME]

Choice: 5
```

```
ana@anas-pc: ~/PCF2/src

Bridge and Torch Game – Save everyone in 17 minutes or less

-----not saved-----bridge-----saved-----

      P2  [10 blocks] 💡 P1      P5 P10

17 min ===== 17 min

Mission: IN PROGRESS

Available choices:
1) P1 [1 min]
2) P5 [5 min]
3) P10 [10 min]
4) P5 + P1 [5 min]
5) P10 + P1 [10 min]
6) P10 + P5 [10 min]
7) [RESTART GAME]
8) [QUIT GAME]

Choice: 1
```

```
ana@anas-pc: ~/PCF2/src

Bridge and Torch Game – Save everyone in 17 minutes or less

-----not saved-----bridge-----saved-----

💡 P1 P2          P5 P10

18 min ===== OVER 17!

Mission: IN PROGRESS

Available choices:
1) P1 [1 min]
2) P2 [2 min]
3) P2 + P1 [2 min]
4) [RESTART GAME]
5) [QUIT GAME]

Choice: 3
```

```
ana@anas-pc: ~/PCF2/src

Bridge and Torch Game – Save everyone in 17 minutes or less

-----not saved-----bridge-----saved-----

          P1 P2 P5 P10

20 min ===== OVER 17!

Mission: FAILED -- try again!

Available choices:
1) [RESTART GAME]
2) [QUIT GAME]

Choice:
```

4.2 State Sequence of Adventurers' Movements

4.2.1 Monad ListDurLog

To support the state sequence functionality, we introduced a new monad, `ListDurLog`, which extends `ListDur` by attaching a log of intermediate states to each branch of the computation. Each branch is a `Duration` wrapping a pair `([State], a)`, where `[State]` accumulates the sequence of states visited along that branch, `a` is the current computed value, and the `Duration` records the total elapsed time.

The `pure` operation embeds a value with zero elapsed time and an empty log, reflecting a computation that has not yet performed any move. The bind operator `(>>=)` sequences two computations by concatenating their logs (`log1 ++ log2`) and summing their durations (`t + t'`), so the full history of intermediate states is built up automatically as the computation progresses, without any explicit bookkeeping at the call sites. `manyLDLChoice` merges the branches of multiple `ListDurLog` computations into a single one, in direct analogy with `manyChoice` from `ListDur`.

This design is inspired by the `LogList` monad from the Knight's Quest exercise.

```
data ListDurLog a = LDL [Duration ([State], a)] deriving Show

remLDL :: ListDurLog a -> [Duration ([State], a)]
remLDL (LDL x) = x

instance Functor ListDurLog where
  fmap f = LDL . map (fmap (\(log, x) -> (log, f x))) . remLDL

instance Applicative ListDurLog where
  pure x = LDL [Duration (0, ([], x))]
  l1 <*> l2 = LDL $ do
    x <- remLDL l1
    y <- remLDL l2
    return $ do (log1, f) <- x; (log2, a) <- y; return (log1 ++ log2, f a)

instance Monad ListDurLog where
  return = pure
  l >>= k = LDL $ do
    Duration (t, (log1, x)) <- remLDL l
    Duration (t', (log2, y)) <- remLDL (k x)
    return $ Duration (t + t', (log1 ++ log2, y))

manyLDLChoice :: [ListDurLog a] -> ListDurLog a
manyLDLChoice = LDL . concatMap remLDL
```

4.2.2 Function playLog

The function `playLog` mirrors the function `play` from the second task but returns a `ListDurLog State` instead of a `ListDur State`, recording the resulting state of each

move in the log.

For a single-adventurer move (`Left a`), the function constructs a branch with the crossing time of `a` as the duration, and stores the resulting state both as the log entry and as the current value. For a two-adventurer move (`Right (a1, a2)`), the duration is the maximum of the two crossing times (since they must cross together at the pace of the slower one), and the resulting state after both adventurers and the lantern have moved is logged in the same way. In both cases, wrapping the result in `LDL` produces a `ListDurLog State` with a single branch, which `playsLog` and `allValidPlaysLog` then compose using the monadic bind.

```
playLog :: Move -> State -> ListDurLog State
playLog (Left a) s =
  LDL [Duration (getTimeAdv a,
    ([mChangeState [Left a, Right ()] s],
    mChangeState [Left a, Right ()] s))]
playLog (Right (a1, a2)) s =
  LDL [Duration (max (getTimeAdv a1) (getTimeAdv a2),
    ([mChangeState [Left a1, Left a2, Right ()] s],
    mChangeState [Left a1, Left a2, Right ()] s))]
```

4.2.3 Function playsLog

The function `playsLog` sequences a list of moves by chaining `playLog` calls through the monadic bind operator. At each step, the bind accumulates the log of visited states and sums the elapsed durations automatically, so neither needs to be managed explicitly.

The base case returns the current state wrapped in `return`, which produces an empty log and zero elapsed time. The initial state is intentionally not included in the log here; it is prepended separately at the display level (in `showPlaysLog` and `showPlaysLog2`) to avoid duplicating it at the concatenation points between consecutive moves.

```
playsLog :: [Move] -> State -> ListDurLog State
playsLog [] s = return s
playsLog (m:ms) s = do s' <- playLog m s
                      playsLog ms s'
```

4.2.4 Function diffStates

The function `diffStates` computes the list of adventurers whose positions differ between two given states. It takes two states `s1` and `s2` as input and returns the list of adventurers that are on different sides of the bridge in each state.

```
diffStates :: State -> State -> [Adventurer]
diffStates s1 s2 = filter(\a -> s1 (Left a) /= s2 (Left a)) [P1, P2, P5, P10]
```

The function filters the list of all adventurers, keeping only those whose position differs between the two states. Since consecutive states in a sequence differ precisely in the adventurers that crossed during the intervening move, `diffStates` effectively identifies who crossed the bridge at each step. This makes it a key auxiliary function for reconstructing and displaying the sequence of moves from a list of states.

4.2.5 Function crossTime

The function `crossTime` computes the time elapsed during the transition between two consecutive states. It takes two states `s1` and `s2` as input and returns the crossing time of that step as an integer.

```
crossTime :: State -> State -> Int
crossTime s1 s2 =
  maximum [ getTimeAdv a | a <- diffStates s1 s2 ]
```

The function first uses `diffStates s1 s2` to identify which adventurers moved between the two states. It then retrieves the crossing time of each of those adventurers using `getTimeAdv` and takes the maximum. This correctly models the constraint that when multiple adventurers cross together, the total time is determined by the slowest one. The result is used during display to accumulate and show the correct elapsed time at each step.

4.2.6 Function showMove'

The function `showMove'` produces a human-readable description of the transition between two consecutive states. It takes two states `s1` and `s2` as input and returns a string describing the move that occurred.

```
showMove' :: State -> State -> String
showMove' s1 s2 = show (diffStates s1 s2) ++ " " ++ dir
  where dir = if s2 (Right ()) then "cross" else "cross back"
```

The function uses `diffStates s1 s2` to determine which adventurers moved, then checks the lantern's position in the resulting state `s2` to determine the direction of travel: if `s2 (Right ())` is `True`, the lantern moved to the far side, meaning the adventurers crossed forward; otherwise they returned. The result is a readable string such as `[P1,P2] cross` or `[P1] cross back`, making the solution trace easy to follow.

4.2.7 Function showPlaysLog

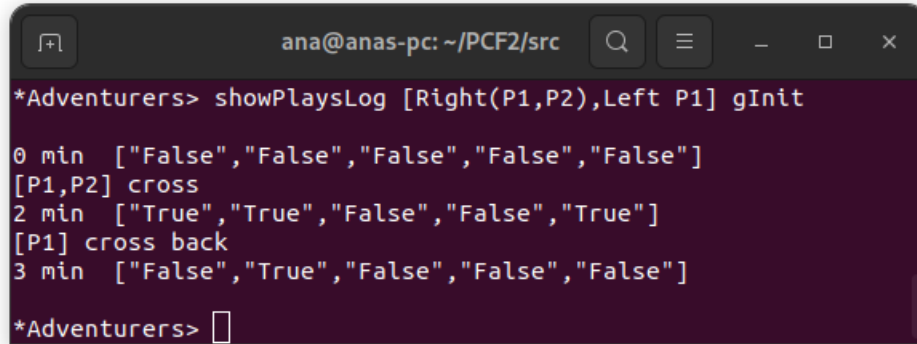
The function `showPlaysLog` takes a list of moves and an initial state and prints the full sequence of states and transitions to the terminal in a textual format.

```
showPlaysLog :: [Move] -> State -> IO ()
showPlaysLog ms s0 =
  case remLDL (playsLog ms s0) of
    [] -> putStrLn "No result."
    (Duration (t, (steps, _)):_) ->
      putStr $ unlines $
        [""] ++ showSteps 0 (s0 : steps) ++ [""]
  where
    showSteps :: Int -> [State] -> [String]
    showSteps _ [] = []
    showSteps t [s] = [show t ++ " min " ++ show s]
    showSteps t (s1:s2:rest) =
      [ show t ++ " min " ++ show s1
```



```
, showMove' s1 s2]
++ showSteps (t + crossTime s1 s2) (s2:rest)
```

The function first runs `playsLog ms s0` to obtain the logged sequence of states, then extracts the first result using `remLDL`. The auxiliary function `showSteps` recursively walks the list of states, printing the elapsed time and state at each step, interleaved with a description of the move that caused each transition (using `showMove'`). The accumulated time is updated at each step using `crossTime`. This produces a readable, step-by-step trace of the entire solution.



```
*Adventurers> showPlaysLog [Right(P1,P2),Left P1] gInit

0 min  ["False","False","False","False","False"]
[P1,P2] cross
2 min  ["True","True","False","False","True"]
[P1] cross back
3 min  ["False","True","False","False","False"]

*Adventurers> 
```

Figure 4.1: Example execution of ‘showPlaysLog’ for a valid move sequence and for a valid initial state.

4.2.8 Function showPlaysLog2

The function `showPlaysLog2` is a variant of `showPlaysLog` that renders the same information using the graphical `drawDurationState` function instead of plain text output.

```
showPlaysLog2 :: [Move] -> State -> IO ()
showPlaysLog2 ms s0 =
  case remLDL (playsLog ms s0) of
    [] -> putStrLn "No result."
    (Duration (t, (steps, _)):_ ) -> drawSteps 0 (s0 : steps)
  where
    drawSteps :: Int -> [State] -> IO ()
    drawSteps _ [] = return ()
    drawSteps t [s] = drawDurationState (Duration (t, s))
    drawSteps t (s1:s2:rest) = do
      drawDurationState (Duration (t, s1))
      putStrLn $ "      \ESC[35m" ++ showMove' s1 s2 ++ "\ESC[0m"
      drawSteps (t + crossTime s1 s2) (s2:rest)
```

Like `showPlaysLog`, it reconstructs the trace from the logged states returned by `playsLog`. However, instead of printing the state as a raw string, it calls `drawDurationState` to render the visual bridge representation at each step. The move description between consecutive states is printed in purple using ANSI escape codes for visual clarity. This provides a richer and more intuitive view of the solution sequence.

```

ana@anas-pc: ~/PCF2/src
*Adventurers> showPlaysLog2 [Right(P1,P2),Left P1] gInit

-----not saved-----bridge-----saved-----
💡 P1 P2 P5 P10  [checkered bridge]
0 min ..... 17 min
[P1,P2] cross
-----not saved-----bridge-----saved-----
P5 P10 [checkered bridge] 💡 P1 P2
2 min ==..... 17 min
[P1] cross back
-----not saved-----bridge-----saved-----
💡 P1 P5 P10 [checkered bridge] P2
3 min ===..... 17 min
*Adventurers> 

```

Figure 4.2: Example execution of ‘showPlaysLog2’ for a valid move sequence and for a valid initial state.

4.2.9 Function allValidPlaysLog

The function `allValidPlaysLog` mirrors `allValidPlays` from the first task but operates over the `ListDurLog` monad, logging the resulting state of each move into the accumulated history.

```

allValidPlaysLog :: State -> ListDurLog State
allValidPlaysLog s = manyLDLChoice (singles ++ pairs)

where singles = [ LDL [Duration
    (getTimeAdv a,
    ([mChangeState [Left a, Right ()] s ],
    mChangeState [Left a, Right ()] s ))]
    | a <- (advsWithLantern s)]

pairs = [ LDL [Duration
    (max (getTimeAdv a1) (getTimeAdv a2),
    ([mChangeState [Left a1, Left a2, Right ()] s ],
    mChangeState [Left a1, Left a2, Right ()] s ))]
    | (a1, a2) <- makePairs (advsWithLantern s)]

```

As in `allValidPlays`, the function generates all single-adventurer and pair moves available from state `s`, using `advsWithLantern` to determine who can cross. The key difference

is that each branch is now wrapped in the LDL constructor and includes the resulting state both as the log entry (the singleton list `[mChangeState ...]`) and as the current value. The function `manyLDLChoice` then merges all branches into a single `ListDurLog State`, representing all valid next moves with their durations and logged states.

4.2.10 Function `execLog`

The function `execLog` extends `exec` from the first task to the `ListDurLog` monad, computing all possible sequences of exactly n moves from a given initial state while accumulating the full sequence of intermediate states.

```
execLog :: Int -> State -> ListDurLog State
execLog 0 s = return s
execLog n s = do s' <- allValidPlaysLog s
                execLog (n-1) s'
```

The structure is identical to `exec`: the base case wraps the current state in the monad with zero duration and an empty log, while the recursive case uses `allValidPlaysLog` to generate all valid next states and chains them through `execLog (n-1)`. The bind operation of `ListDurLog` automatically accumulates both the total duration and the log of visited states across all recursive calls, so the final result contains, for each possible execution path, the full history of states alongside the total elapsed time.

4.2.11 Function `bestPaths`

The function `bestPaths` collects all optimal solution paths: sequences of states that start at `gInit`, end at `gFinal`, and complete within 17 minutes, explored up to a given maximum number of moves.

```
bestPaths :: Int -> [[State]]
bestPaths maxMoves =
  [ log
  | n <- [0..maxMoves]
  , Duration (t, (log, s)) <- remLDL (execLog n gInit)
  , s == gFinal
  , t <= 17
  ]
```

The function iterates over all move counts from 0 to `maxMoves` and runs `execLog n gInit` for each, extracting the logged branches via `remLDL`. Each branch contains the total elapsed time `t`, the accumulated state history `log`, and the final state `s`. Only branches where the final state equals `gFinal` and the total time does not exceed 17 minutes are kept. The result is a list of state sequences, each representing a valid and optimal path through the bridge-crossing problem.

4.2.12 Function `statesToMoves`

The function `statesToMoves` reconstructs the list of moves that produced a given sequence of states. It takes a list of states and returns the corresponding list of `Move` values.

```

statesToMoves :: [State] -> [Move]
statesToMoves [] = []
statesToMoves [_] = []
statesToMoves (s1:s2:rest) =
  case diffStates s1 s2 of
    [a]      -> Left a : statesToMoves (s2:rest)
    [a1,a2]  -> Right (a1,a2) : statesToMoves (s2:rest)
    _        -> statesToMoves (s2:rest)

```

The function processes consecutive pairs of states: for each pair (`s1`, `s2`), it uses `diffStates` to determine which adventurers moved. If exactly one adventurer changed side, the move is a single crossing (`Left a`); if two adventurers changed side, the move is a pair crossing (`Right (a1, a2)`). Any other case is skipped. This function bridges the gap between the state-sequence representation produced by `bestPaths` and the move-list representation expected by `showPlaysLog` and `showPlaysLog2`.

4.2.13 Function `printBestPath`

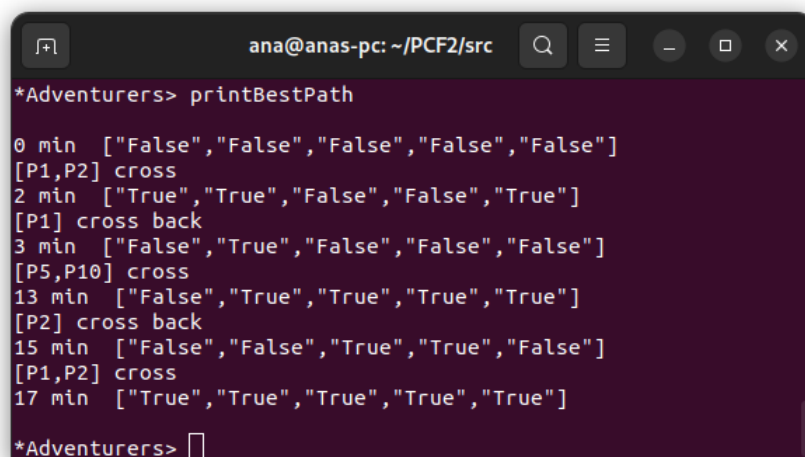
The function `printBestPath` prints the first optimal solution found by `bestPaths` as a textual step-by-step trace.

```

printBestPath :: IO ()
printBestPath = showPlaysLog (statesToMoves (gInit: (head (bestPaths 5)))) gInit

```

It calls `bestPaths 5` to retrieve all valid solutions within 5 moves and 17 minutes, selects the first one with `head`, and prepends `gInit` to reconstruct the full state sequence starting from the initial configuration. It then uses `statesToMoves` to convert this sequence into a move list, which is passed to `showPlaysLog` together with the initial state to render the complete trace in plain text.



```

ana@anas-pc: ~/PCF2/src
*Adventurers> printBestPath

0 min  ["False","False","False","False","False"]
[P1,P2] cross
2 min  ["True","True","False","False","True"]
[P1] cross back
3 min  ["False","True","False","False","False"]
[P5,P10] cross
13 min ["False","True","True","True","True"]
[P2] cross back
15 min ["False","False","True","True","False"]
[P1,P2] cross
17 min ["True","True","True","True","True"]

*Adventurers> 

```

Figure 4.3: Evaluation of the function `printBestPath` in GHCi.

4.2.14 Function `printBestPath2`

The function `printBestPath2` serves the same purpose as `printBestPath`, but renders the solution using the graphical display provided by `showPlaysLog2`.

```
printBestPath2 :: IO ()
printBestPath2 = showPlaysLog2 (statesToMoves (gInit: (head (bestPaths 5)))) gInit
```

It follows exactly the same logic as `printBestPath` — selecting the first optimal path from `bestPaths 5`, reconstructing the move list via `statesToMoves`, and passing it to the display function — but delegates rendering to `showPlaysLog2`, which draws the bridge diagram at each step with ANSI colors. This makes the solution trace visually consistent with the interactive game interface.

```
*Adventurers> printBestPath2

-----not saved-----bridge-----saved-----
💡 P1 P2 P5 P10  ██████████
0 min ..... 17 min

[P1,P2] cross

-----not saved-----bridge-----saved-----
P5 P10  ██████████ 💡 P1 P2
2 min ==..... 17 min

[P1] cross back

-----not saved-----bridge-----saved-----
💡 P1 P5 P10  ██████████ P2
3 min ==..... 17 min

[P5,P10] cross

-----not saved-----bridge-----saved-----
P1  ██████████ 💡 P2 P5 P10
13 min =====... 17 min

[P2] cross back

-----not saved-----bridge-----saved-----
💡 P1 P2  ██████████ P5 P10
15 min =====.. 17 min

[P1,P2] cross

-----not saved-----bridge-----saved-----
██████████ 💡 P1 P2 P5 P10
17 min ===== 17 min

*Adventurers> 
```

Figure 4.4: Evaluation of the function `printBestPath2` in GHCi.

4.3 Conclusions

In this task, we implemented two independent functionalities that we considered both useful and interesting extensions to the base assignment.

The interactive game using the IO monad provides a hands-on way to explore the bridge-crossing problem. Rather than inspecting raw output from functions like `exec` or `leq17`, the player can make decisions step by step and directly observe how each choice affects the remaining time and the positions of the adventurers. The game also provides immediate feedback on success or failure, making it clear whether the chosen sequence of moves satisfies the 17-minute constraint. This brings the abstract model to life in a way that purely functional output cannot.

The state sequence functionality addresses a different limitation: the functions from the previous tasks only expose the final state and total duration of an execution, with no visibility into the intermediate steps. By introducing the `ListDurLog` monad and the associated functions, it becomes possible to trace the full path from the initial state to the goal, inspect every intermediate state, and confirm that the constraints of the problem are respected at each move. Functions like `printBestPath` and `printBestPath2` then use this infrastructure to automatically find and display an optimal solution.

One of the most striking observations across this task is how much of the new code is structurally identical to code already written in the previous tasks, despite operating over a different monad. The clearest example is the pair `execLog` and `exec`:

```
exec :: Int -> State -> ListDur State
exec 0 s = return s
exec n s = do s' <- allValidPlays s
             exec (n-1) s'

execLog :: Int -> State -> ListDurLog State
execLog 0 s = return s
execLog n s = do s' <- allValidPlaysLog s
               execLog (n-1) s'
```

The two functions are letter-for-letter identical in structure. The only differences are the return type (`ListDur` vs `ListDurLog`) and the helper called (`allValidPlays` vs `allValidPlaysLog`). The recursive logic, the base case, and the `do` notation are completely unchanged, because the bind operation handles all the monad-specific bookkeeping — duration accumulation in `ListDur`, and duration accumulation plus log concatenation in `ListDurLog` — invisibly and automatically.

The same pattern appears in `playsLog` vs `plays` from the second task:

```
plays :: [Move] -> State -> DistDur State
plays []      s = return s
plays (m:ms) s = do s' <- play m s
                   plays ms s'

playsLog :: [Move] -> State -> ListDurLog State
```

```
playsLog []      s = return s
playsLog (m:ms) s = do s' <- playLog m s
                    playsLog ms s'
```

`plays` sequences moves over a probability distribution, accumulating probabilities and durations. `playsLog` sequences moves over a non-deterministic list, accumulating durations and state logs. Yet both are written in exactly the same way, because the monad abstracts over those differences entirely.

This recurring pattern illustrates one of the fundamental advantages of monadic programming: the same high-level control flow — sequencing steps and accumulating effects — can be expressed once and reused across entirely different computational contexts simply by changing the monad. The programmer writes the logic; the monad handles the effects.

Chapter 5

Conclusion

In this report, I completed the three tasks related to the modelling and analysis of a cyber-physical system using monads.

The successful completion of these tasks demonstrates an understanding of the concepts and techniques associated with monads and their application in cyber-physical systems.

Each task explored monads from a different angle: the first used non-determinism and duration to model and verify the system; the second introduced probability to reason about uncertain crossing times; and the third applied the `IO` monad for interactivity and a logging monad to reconstruct solution traces. Together, they show how the same monadic mechanism adapts naturally to very different computational contexts.

I enjoyed this assignment because it gave me hands-on experience with monads, helping me understand that they are quite useful and not as intimidating as they initially appear.